

Processing Accelerators

MEI/MCA/MMC/MIA

Agenda 19/10/2026:
1. vector/matrix processing
2. hardware support & performance vs precision
3. programming challenges

2026/27

J.L. Sobral

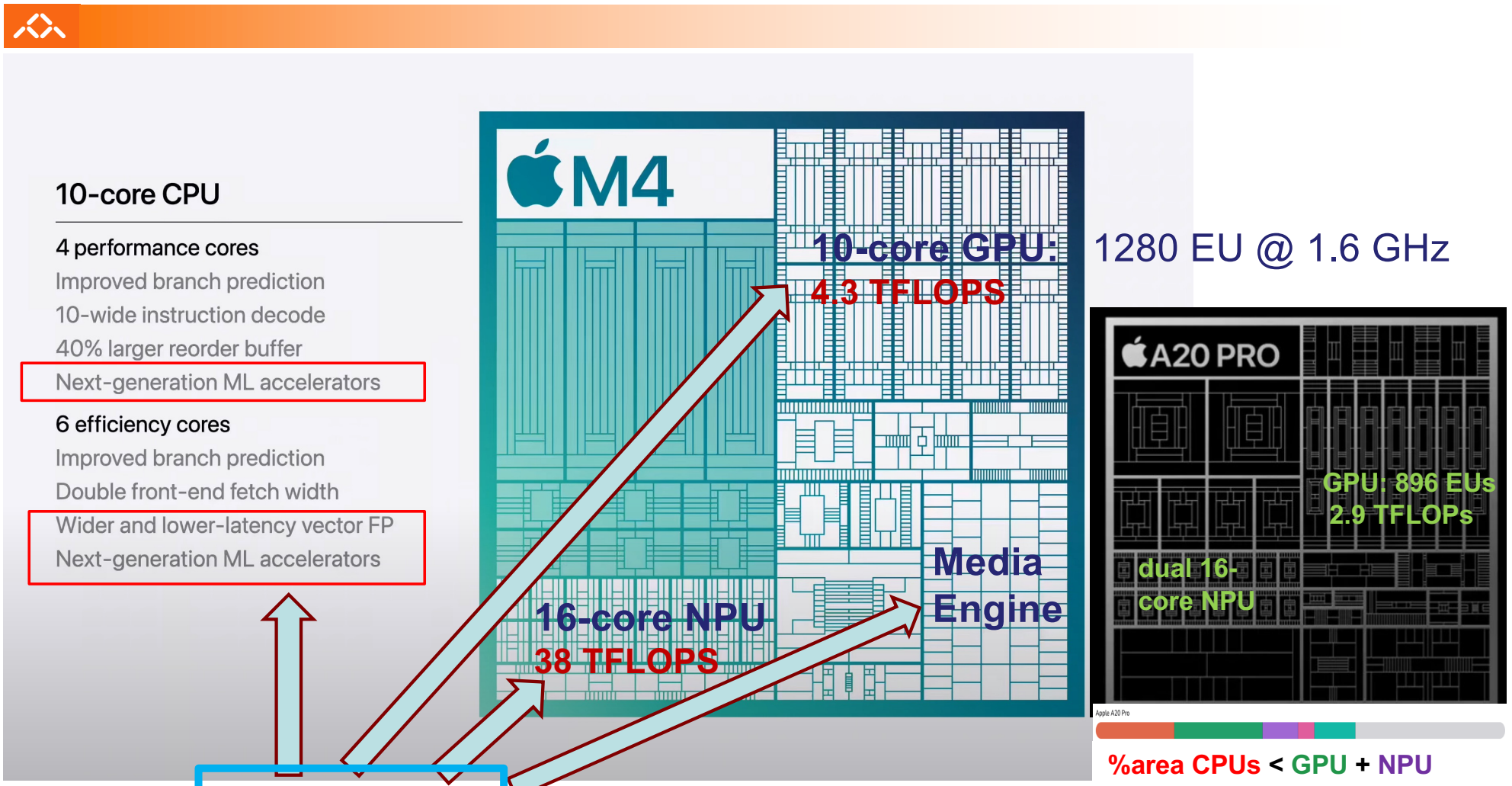
What is a Processing accelerator?



"**Processing accelerators** are specialized hardware components that perform specific computational tasks much faster and more efficiently than a general-purpose CPU. Common examples include Graphics Processing Units (GPUs) for parallel tasks like AI and image processing, Tensor Processing Units (TPUs) for AI workloads, and Field-Programmable Gate Arrays (FPGAs) which offer flexibility. These accelerators are crucial for modern applications like AI, machine learning, and high-performance computing that require more speed and power than traditional processors can provide alone."

Vista geral IA (google)

SoC Apple M4: Laptop or HPC system?



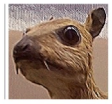
Parallel + specialized HW: We will also discuss how modern systems can utilize specialized processing units to achieve high levels of efficiency

”A fotografia computacional refere-se a técnicas de captura e processamento de imagem digital que utilizam computação, inteligência artificial (IA) e algoritmos em vez de depender exclusivamente de processos ópticos convencionais. O seu **principal objetivo é superar as limitações físicas das lentes e dos sensores fotográficos mais pequenos** (como os dos smartphones), permitindo gerar imagens polidas que rivalizam ou ultrapassam o desempenho do hardware tradicional.”

Stencil examples

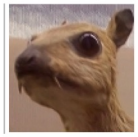
- Convolução para melhorar a qualidade de uma imagem (máscaras de 3x3, 5x5 ou 7x7)

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$



Identity

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$



- Algoritmo:

```
for each image row in input image:
   for each pixel in image row:

      set accumulator to zero

      for each kernel row in kernel:
         for each element in kernel row:

            if element position  corresponding* to pixel position then
               multiply element value  corresponding* to pixel value
               add result to accumulator
            endif

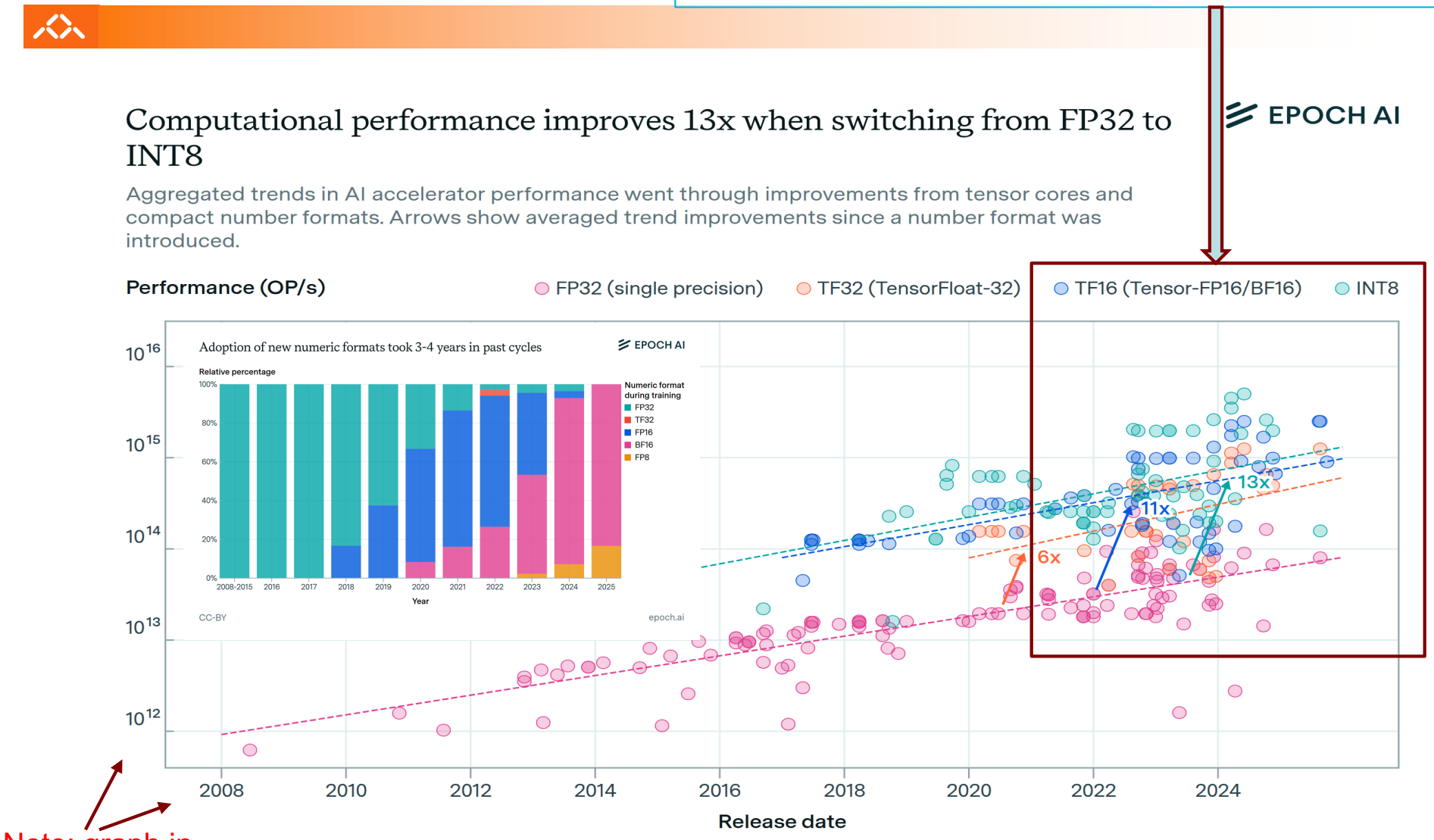
      set output image pixel to accumulator
```

JLSobral, Parallel Algorithms, MCA, UMinho, 2025/2026

- **Modo Retrato (Portrait Mode):** Utiliza mapeamento de profundidade e inteligência artificial para segmentar o sujeito principal do fundo, aplicando um efeito de desfoque (*bokeh*) artificialmente.
- **High Dynamic Range (HDR):** Combina instantaneamente várias exposições da mesma cena (uma subexposta para o céu brilhante e outra superexposta para as sombras) para criar uma imagem final equilibrada e rica em detalhes.
- **Modo Noite (Low Light / Night Sight):** Executa o empilhamento de imagens capturadas sequencialmente em frações de segundo para reduzir o ruído digital e maximizar a captação de luz sem necessitar de um tripé.

Performance vs Precision

Lower FP precision => higher performance



Note: graph in lin-log scale!

NVidia GeForce RTX 50 series

Model	Clock Rate Base (Boost) (MHz)	Clock Rate Memory (MHz)	Core Config (PEs)	Cache L1 per SM (KB)	Cache L2 (MB)	Memory Size (GB)	Memory Bus Width (bit)	Memory Bandwidth (GB/s)	Processing Power TFLOPS FP64 (1:64)	Processing Power TFLOPS FP32	Processing Power TFLOPS FP16 (1:1)	Tensor (AI) Dense FP16 (1:1)	Tensor (AI) Sparse FP16 (2:4) Dense FP8 (1:1)	Tensor (AI) Sparse FP8 (2:4) Dense FP4 (1:1)	Tensor (AI) Sparse FP4 (2:4)
GeForce RTX 5090 D V2	2017 (2407)	1750	21760	128	96	24	384	1344	1.373 (1.637)	1.373 (1.637)	87.780 (104.753)	351.119 (419.011)	702.239 (838.021)	1404.477 (1676.042)	2375.000 (2375.000)
GeForce RTX 5090	2017 (2407)	1750	21760	128	92	32	512	1792	1.373 (1.637)	1.373 (1.637)	87.780 (104.753)	351.119 (419.011)	702.239 (838.021)	1404.477 (1676.042)	2808.955 (3352.084)
GeForce RTX 5080	2295 (2617)	1875	10752	128	64	16	256	960	0.771 (0.879)	0.771 (0.879)	49.352 (56.276)	197.407 (225.104)	394.813 (450.208)	789.627 (900.415)	1579.254 (1800.831)
GeForce RTX 5070	2325 (2512)	1750	6144	128	48	12	192	672	0.446 (0.482)	0.446 (0.482)	28.430 (30.717)	114.278 (123.470)	228.557 (246.940)	457.114 (493.879)	914.227 (987.759)
GeForce RTX 5060	2280 (2445)	1750	3840	128	32	8	128	448	0.274 (0.300)	0.274 (0.300)	17.510 (18.177)	70.042 (76.708)	140.083 (153.416)	280.166 (306.831)	560.333 (613.663)
GeForce RTX 5050	2317 (2572)	2000	2560	128	24	8	128	320	0.185 (0.205)	0.185 (0.205)	11.863 (13.169)	47.452 (52.675)	94.904 (105.350)	189.809 (210.699)	379.617 (421.398)

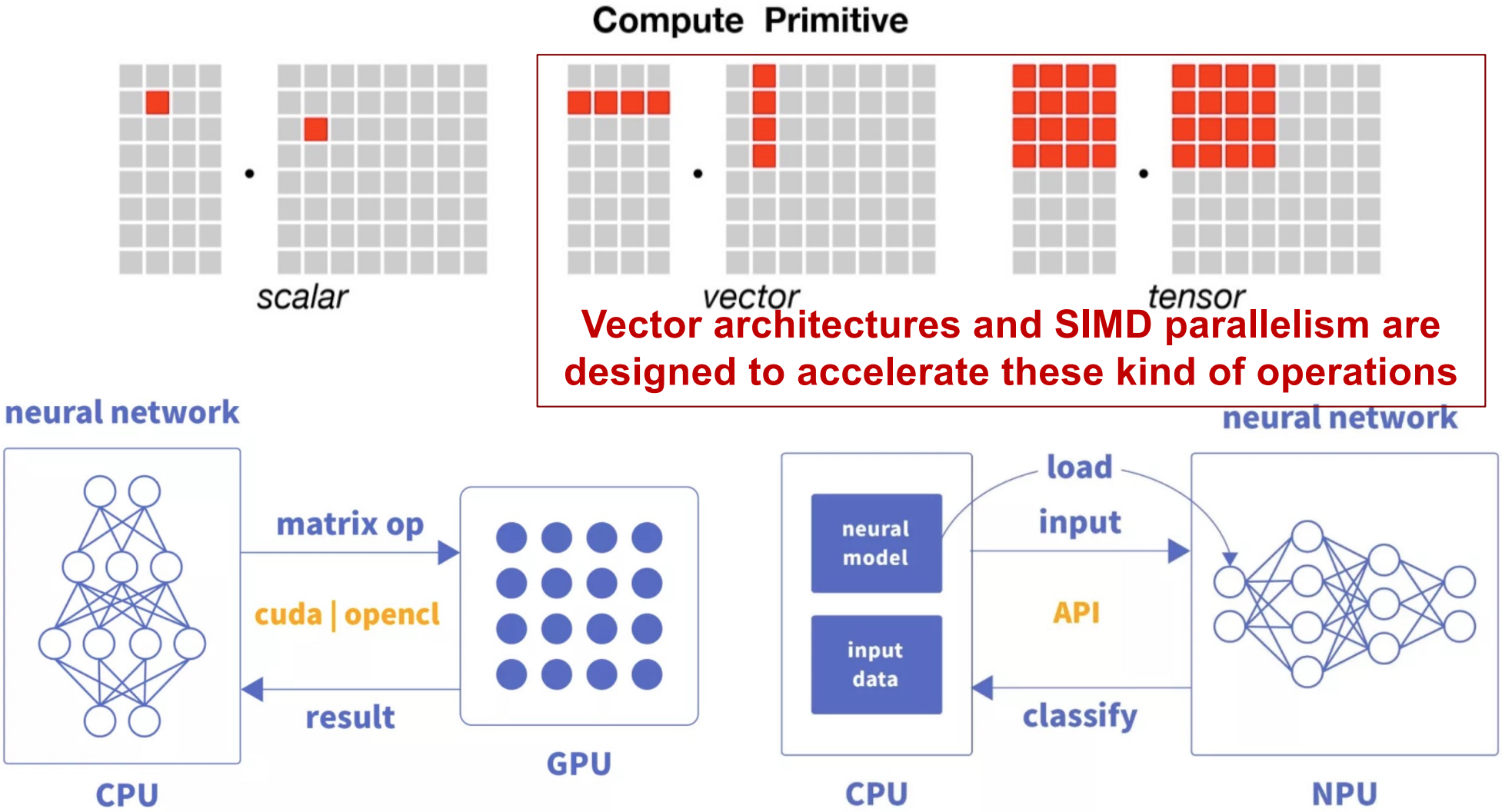
2000x

peak FP16 performance:
#PEs x IPC x Clock Rate =
21760 x 2 (FMA) x 2017 x 10^6

vector processing
FP64/FP32/FP16

matrix processing
FP16/FP8/FP4

Compute primitive (smallest unit) *in CPU, GPU and TPU/NPU*



Vector Architectures (& SIMD parallelism)

- Basic idea:
  - Read sets of data elements *(gather from memory)* into “vector registers”
  - Operate on those registers
  - Disperse the results back into memory *(scatter)*
- Registers are controlled by the compiler
  - Used to hide memory latency
  - Leverage memory bandwidth

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

SIMD Parallelism (1)

- Key principle underlying:
  - Vector architectures
  - SIMD extensions in modern CPUs ***(focus of these slides)***
  - Graphics Processor Units (GPUs) *(in another set of slides)*

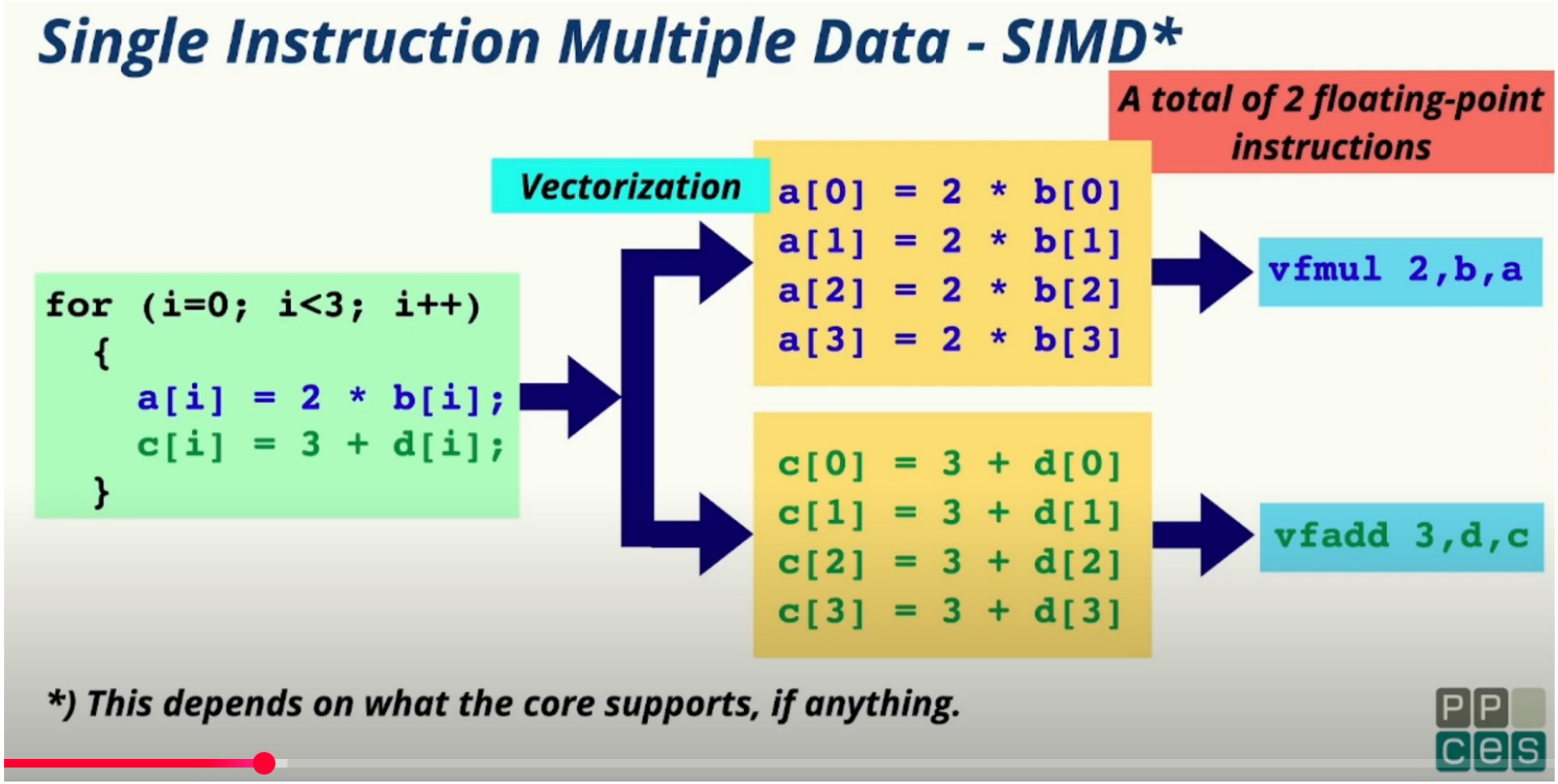
Flynn's Taxonomy of Computers *

		Data Streams	
		Single	Multiple
Instruction Streams	Single	**SISD**: Intel Pentium 4	**SIMD**: AVX instructions in x64_x86 ISA
	Multiple	**MISD**: No examples today	**MIMD**: Multicore devices

** Mike Flynn, "Very High-Speed Computing Systems", Proc. of IEEE, 1966*

MORGAN KAUFMANN

Copyright © 2019, Elsevier Inc. All rights Reserved

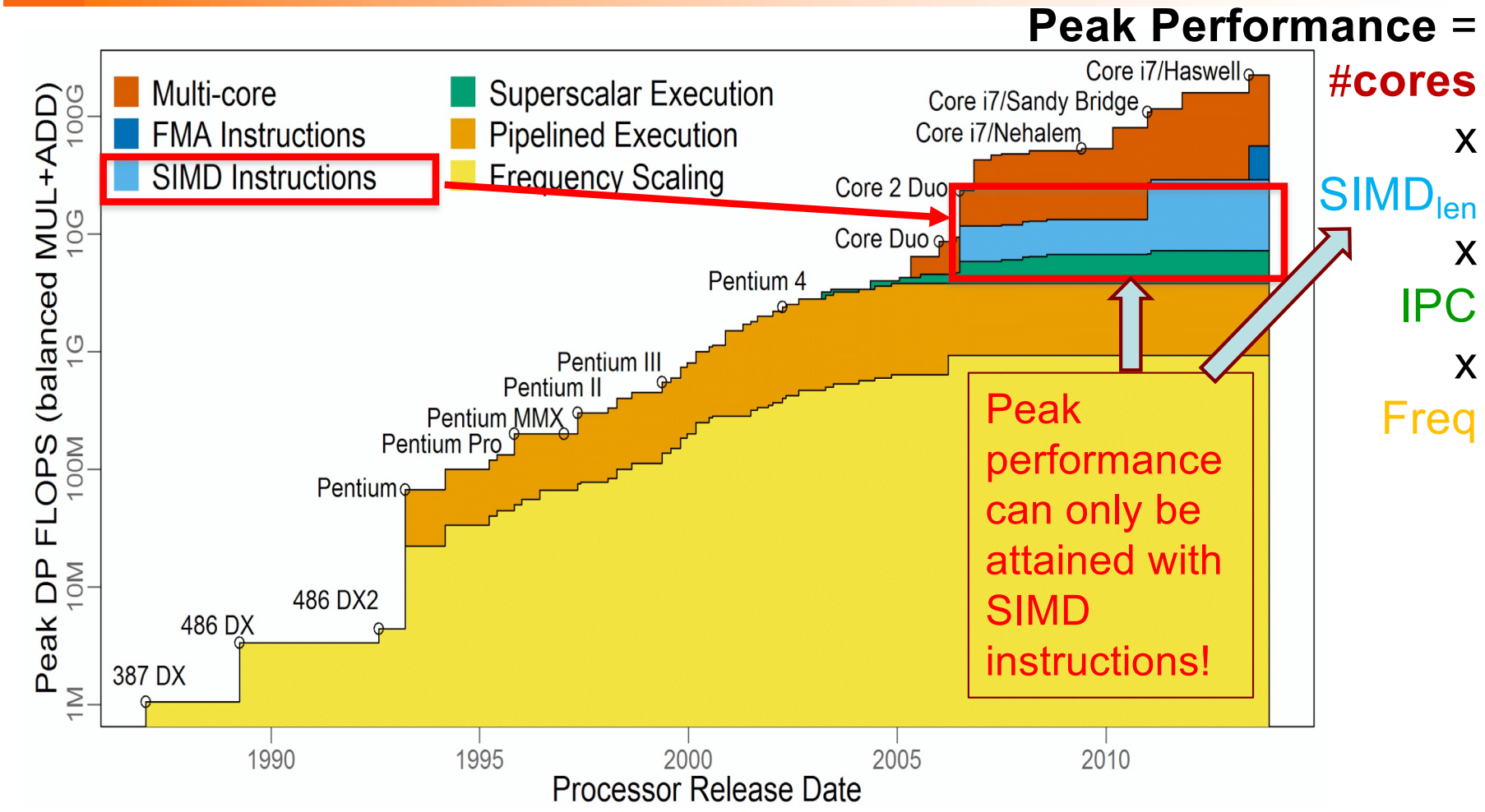

Single Instruction Multiple Data - SIMD*
A total of 2 floating-point instructions
Vectorization
for (i=0; i<3; i++)
{
a[i] = 2 * b[i];
c[i] = 3 + d[i];
}
a[0] = 2 * b[0]
a[1] = 2 * b[1]
a[2] = 2 * b[2]
a[3] = 2 * b[3]
vfmul 2,b,a
c[0] = 3 + d[0]
c[1] = 3 + d[1]
c[2] = 3 + d[2]
c[3] = 3 + d[3]
vfadd 3,d,c
*) This depends on what the core supports, if anything.

SIMD Parallelism (3)

- SIMD architectures can exploit significant data-level parallelism for:
  - Matrix-oriented scientific computing
  - Media-oriented image and sound processors
  - Machine learning algorithms
- SIMD is more energy efficient than MIMD
  - Only needs to fetch one instruction per data operation
  - Makes SIMD attractive for personal mobile devices
- SIMD allows programmer to continue to think sequentially

Copyright © 2019, Elsevier Inc. All rights Reserved

Evolution of Peak performance

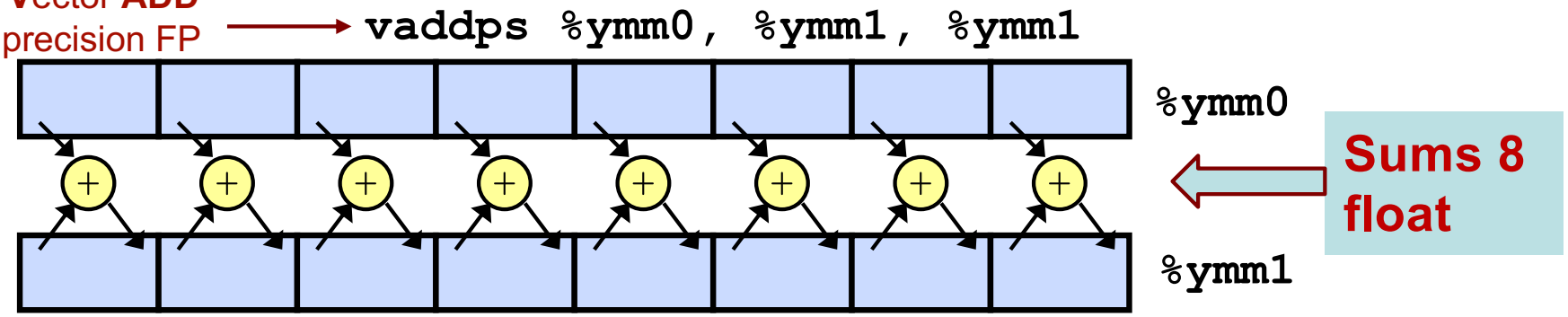


Texe (ideal) = #I$_{vect}$ x CPI x Tcc / **#cores** (#I$_{vect}$ = #I/SIMD$_{len}$)



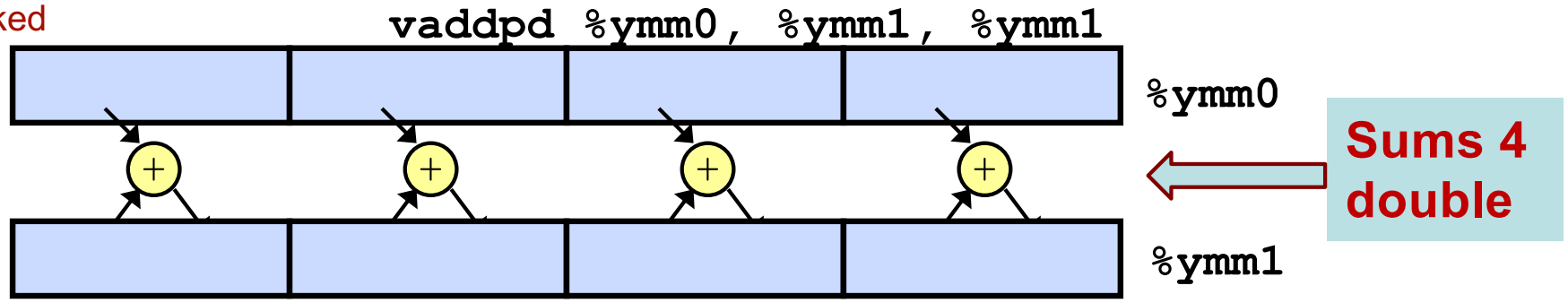
- SIMD operation over **float** (each register contains 8 floats)

Intel AVX ISA: Vector ADD
Packed Single precision FP



- SIMD operation over **double** (each register contains 4 double)

Vector Add Packed
Double precision



Note: in both examples we have 256-bit vector registers

Vector Programming (& challenges)

- Compilers are a key element to give hints on whether a code section will vectorize or not

- Check if loop iterations have data dependencies and/or `if…then` statements, otherwise vectorization is compromised

- Vector architectures have a too high cost, but simpler variants are currently available on off-the-shelf devices, as SIMD extensions to the scalar PUs; **however**:
  - most do not support **non-unit stride**
    => care must be taken in the design of data structures (data layout)
  - same applies for **mask register**,
  - **gather-scatter...**

No data dependencies allowed
- Vector prog. applies only to data-parallelism (e.g. map pattern)

Deal with control dependencies
- common solutions:
  - predication => additional work
  - mask registers => unused PEs

Support for arbitrary vector lengths
- when the vector registers size does not match the vector length

Maximize data locality
- maximize spatial locality (next slides)
- data alignment in memory

NOTE: SIMD consumes much more data per instruction!

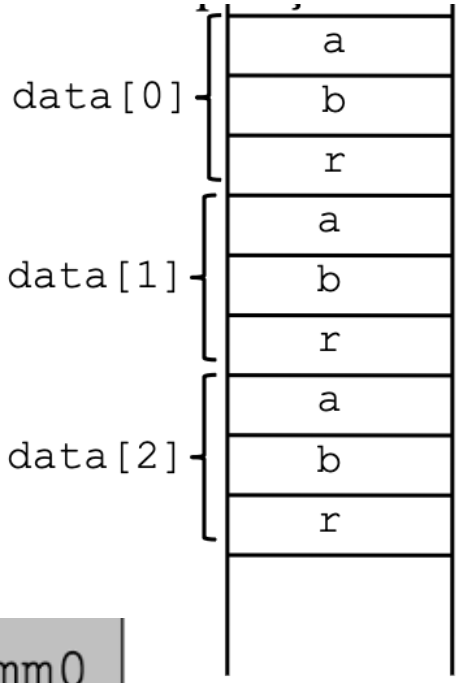
MORGAN KAUFMANN
Copyright © 2012, Elsevier Inc. All rights reserved.

Data layout: AoS vs. SoA (1)

A typical C code program defines the following data layout as an Array of Structures (AoS):

```
struct {
  float a, b, r;
} data[100];
```

This array will be stored in memory like this:



Now consider this instruction:

```
movaps (%eax, %ecx, 4), %xmm0
```

It expects to find a vector of 4 floats in consecutive memory cells: namely `a[0],a[1],a[2],a[3]` but it finds a non-unit stride…

Conclusion: the compiler will not vectorize this code!

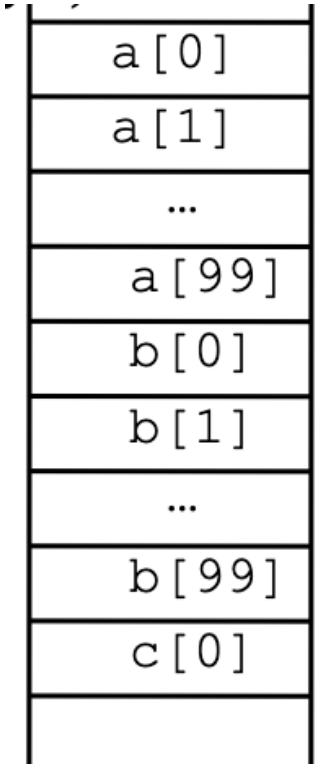
Data layout: AoS vs. SoA (2)

Solution

The C code should redefine the
data layout as a Structure of Arrays (SoA):

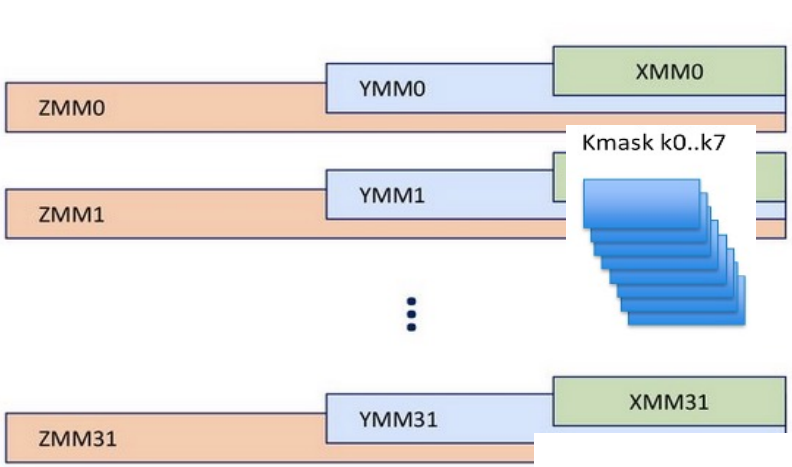
```
struct {
   float a[100], b[100], r[100];
} data;
```

Now the structure of arrays is
stored in memory like this:

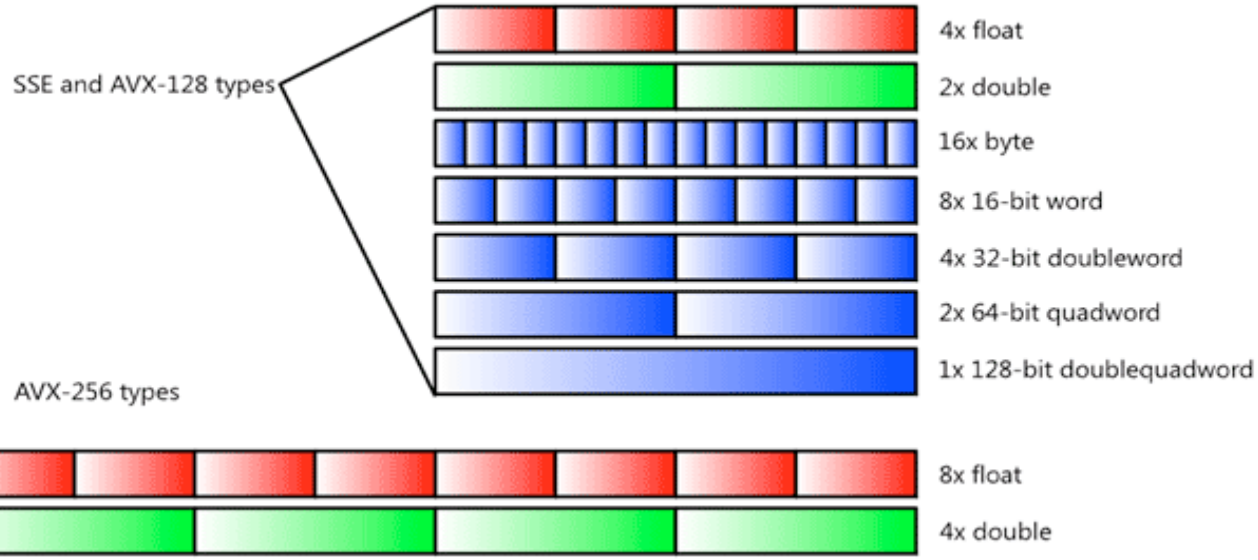


Conclusion: the compiler now vectorizes that code!

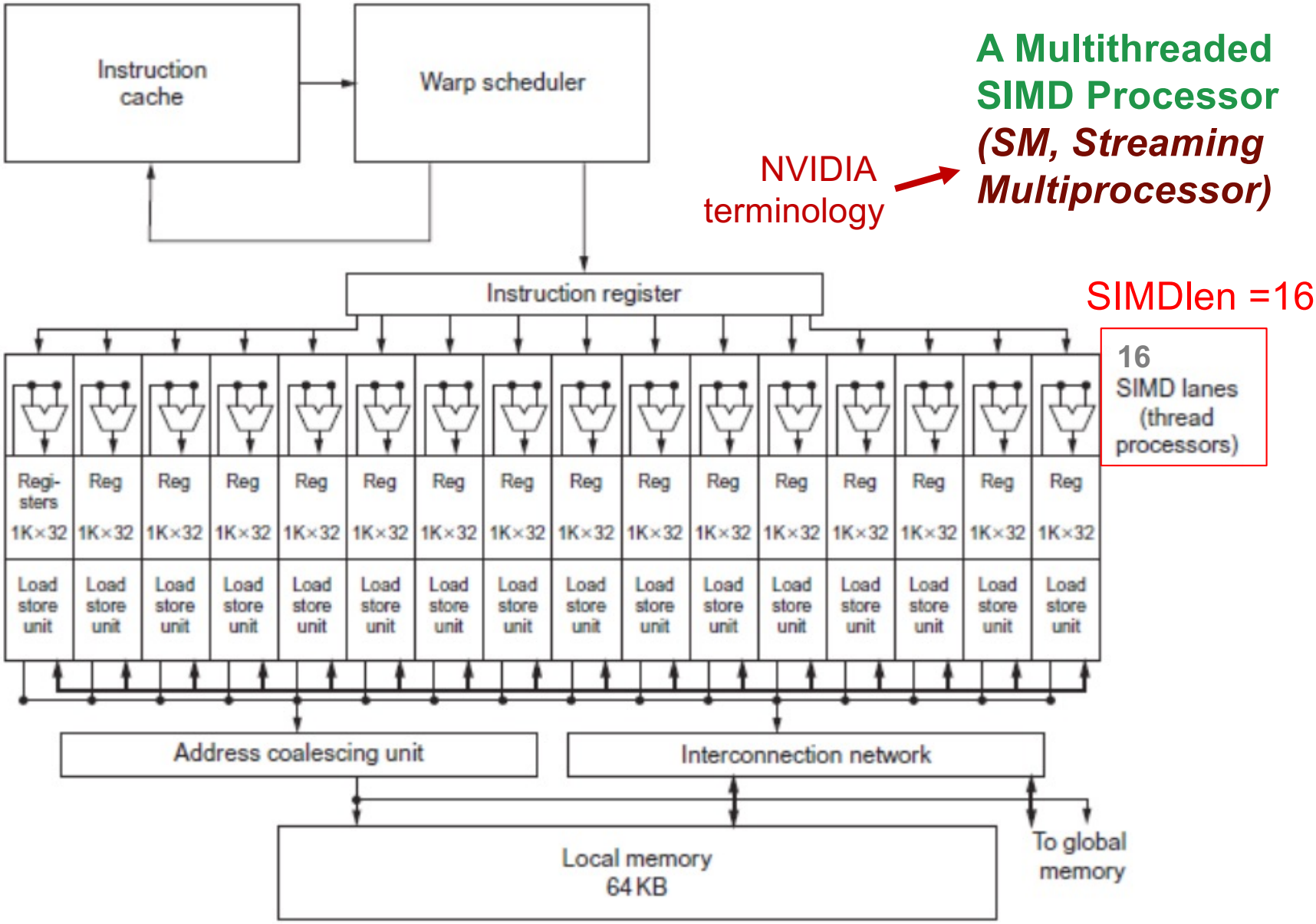
Registers for vector processing in Intel x64



- XMM0 – XMM15
  - 128-bit registers
  - SSE
- YMM0 – YMM15
  - 256-bit registers
  - AVX, AVX2
- ZMM0 – ZMM31
  - 512-bit registers
  - AVX-512

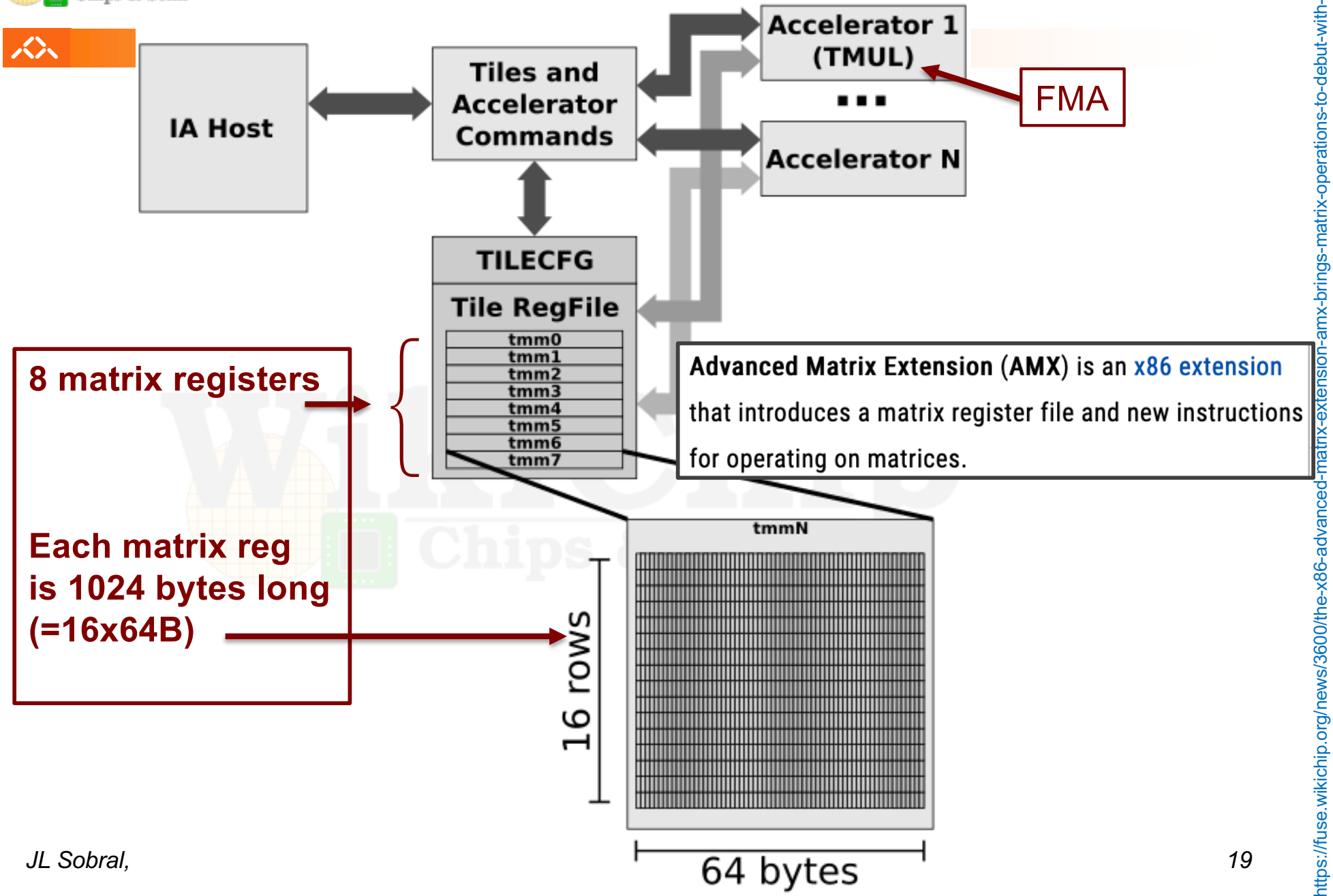


GPU Organization *(NVIDIA)*



MORGAN KAUFMANN
Copyright © 2019, Elsevier Inc. All rights Reserved

Intel Advanced Matrix Extension (AMX)



https://fuse.wikichip.org/news/3600/the-x86-advanced-matrix-extension-amx-brings-matrix-operations-to-debut-with-sapphire-rapids/